The main issue with Around Aliases, however, is that they are a form of Monkeypatching. Like all Monkeypatches, they can break existing code that wasn't expecting the method to change. For this reason, Ruby 2.0 introduced not one, but two additional ways to wrap additional functionality around an existing method.

More Method Wrappers

In *Refinements, on page 36*, you learned that a *Refinement (36)* works like a patch of code that has been slapped directly over a class. However, Refinements have one additional feature that enables you to use them in place of *Around Aliases (134)*: if you call `super` from a refined method, you will call the original, unrefined method. Here comes an example:

class_definitions/wrapper_refinement.rb

```
module StringRefinement
  refine String do
    def length
      super > 5 ? 'long' : 'short'
    end
  end
end

using StringRefinement

"War and Peace".length      # => "long"
```

The code above refines the `String` class to wrap additional functionality around its `length` method. Like other Refinements, this *Refinement Wrapper* applies only until the end of the file (or, in Ruby 2.1, the module definition). This makes it generally safer than the equivalent Around Alias, which applies everywhere.

Spell: Refinement Wrapper

Finally, you have a third way of wrapping a method: you can use `Module#prepend`, which you might remember from *Modules and Lookup, on page 30*. `Module#prepend` works a bit like `include`, but it inserts the module *below* the includer in the chain of ancestors, rather than above it. This means that a method in a prepended module can override a method in the includer and call the non-overridden version with `super`:

class_definitions/wrapper_prepend.rb

```
module ExplicitString
  def length
    super > 5 ? 'long' : 'short'
  end
end
```